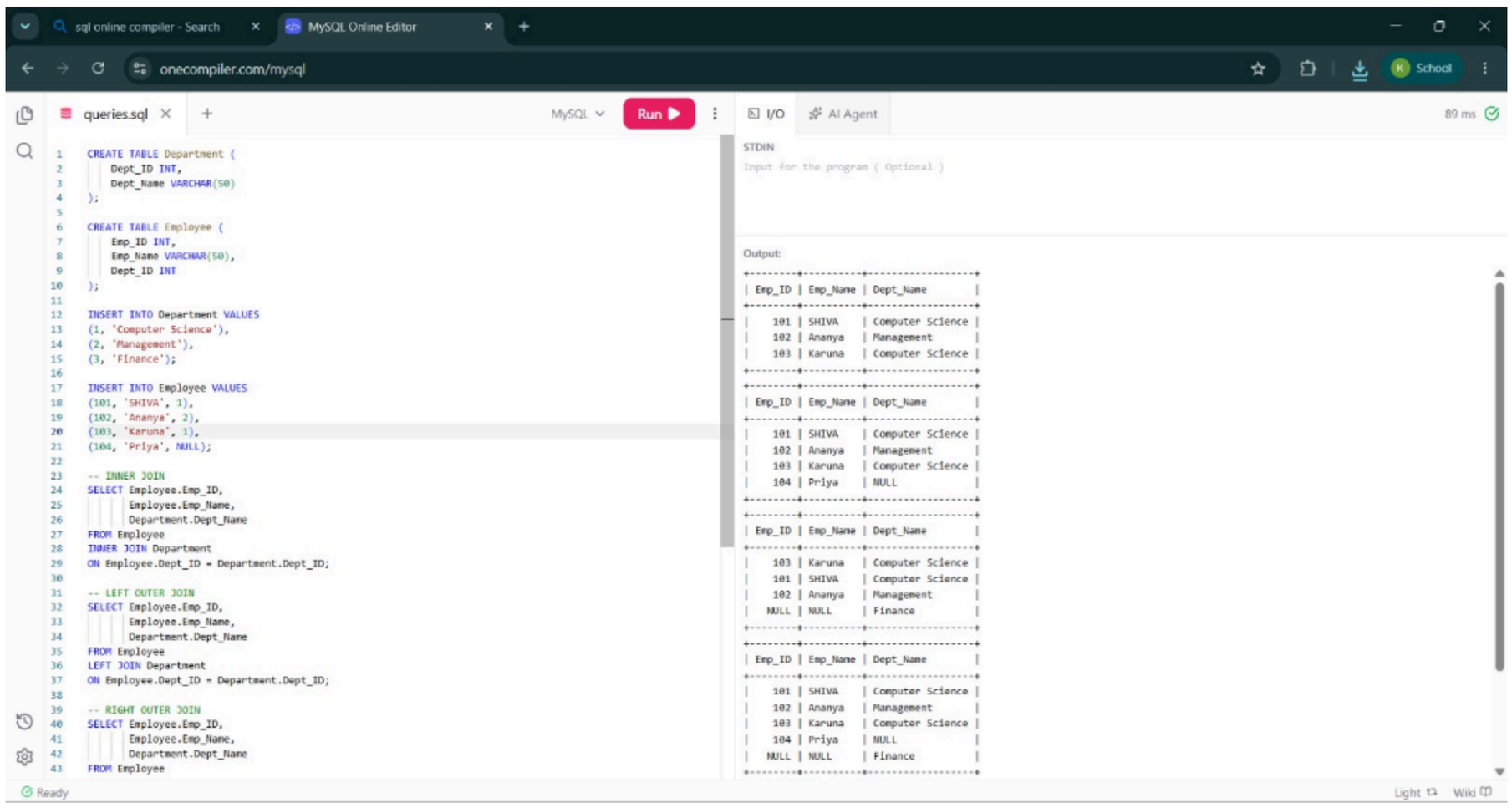
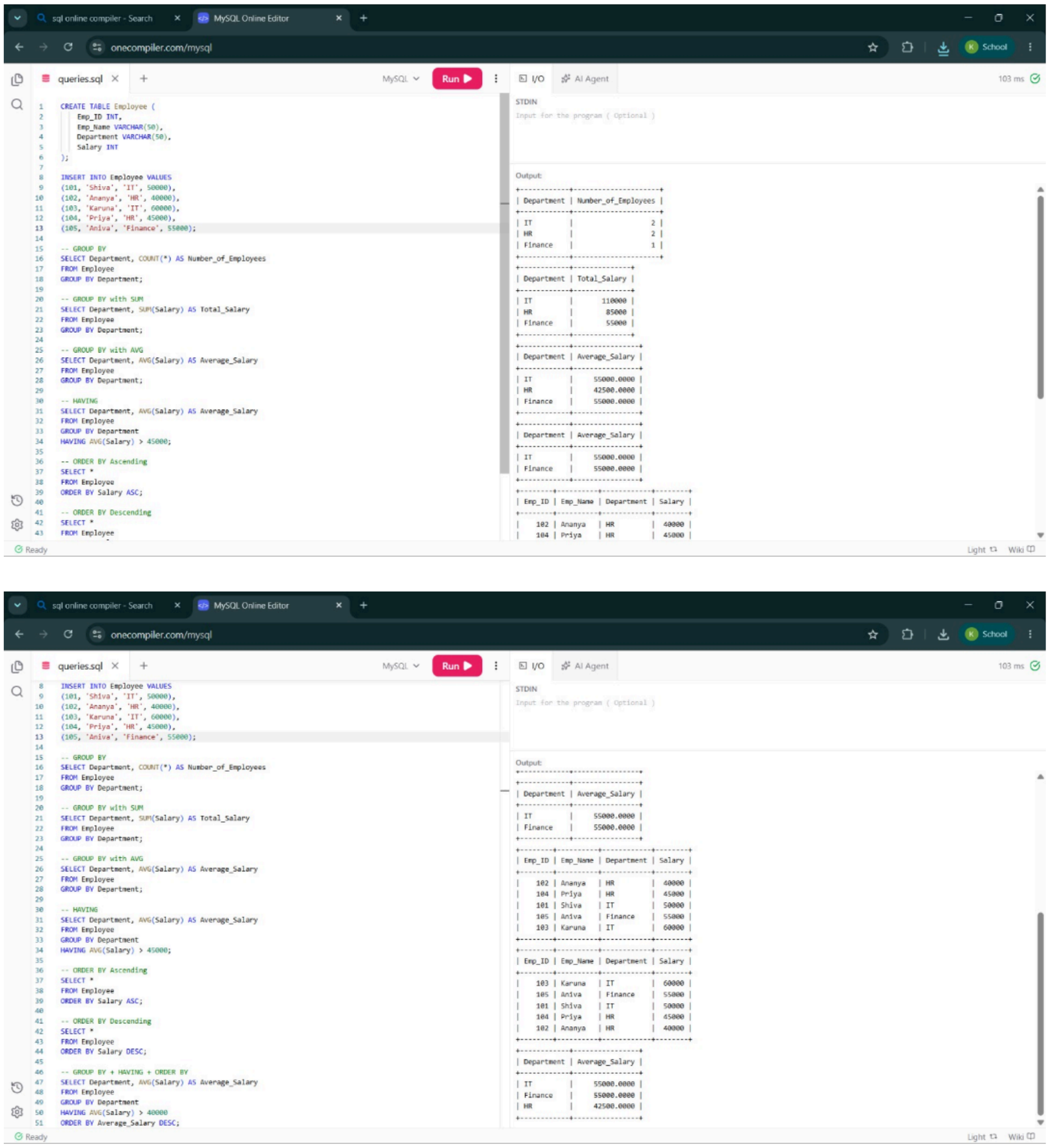
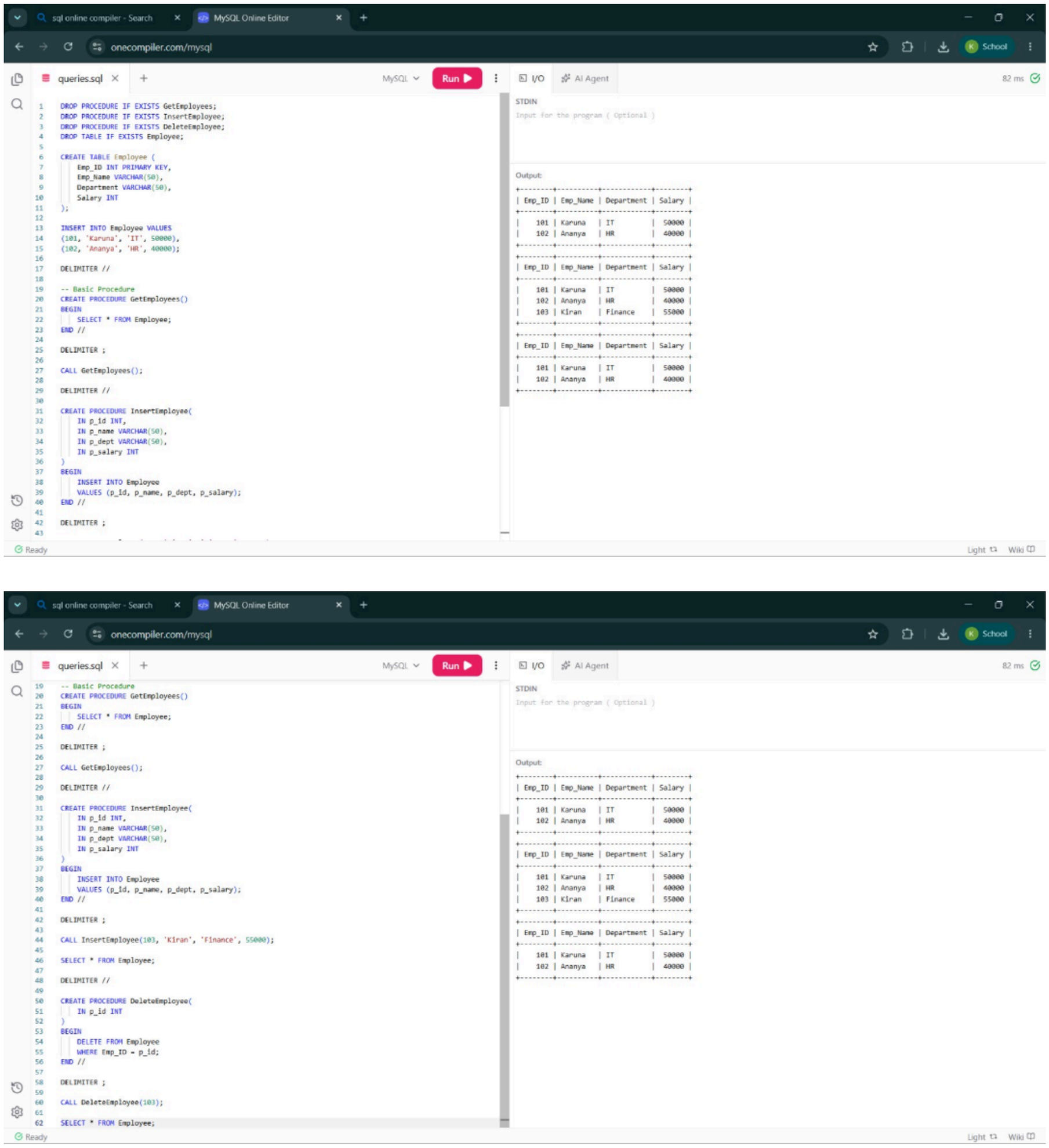
LAB 7:


sql online compiler - Search
MySQL Online Editor
onecompiler.com/mysql
School
queries.sql
MySQL
Run
I/O
AI Agent
89 ms
STDIN
Input for the program (Optional)
Output:
CREATE TABLE Department (
Dept_ID INT,
Dept_Name VARCHAR(50)
);
CREATE TABLE Employee (
Emp_ID INT,
Emp_Name VARCHAR(50),
Dept_ID INT
);
INSERT INTO Department VALUES
(1, 'Computer Science'),
(2, 'Management'),
(3, 'Finance');
INSERT INTO Employee VALUES
(101, 'SHIVA', 1),
(102, 'Ananya', 2),
(103, 'Karuna', 1),
(104, 'Priya', NULL);
-- INNER JOIN
SELECT Employee.Emp_ID,
Employee.Emp_Name,
Department.Dept_Name
FROM Employee
INNER JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;
-- LEFT OUTER JOIN
SELECT Employee.Emp_ID,
Employee.Emp_Name,
Department.Dept_Name
FROM Employee
LEFT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;
-- RIGHT OUTER JOIN
SELECT Employee.Emp_ID,
Employee.Emp_Name,
Department.Dept_Name
FROM Employee
RIGHT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;
-- FULL OUTER JOIN
-- MySQL does not directly support FULL OUTER JOIN.
SELECT Employee.Emp_ID,
Employee.Emp_Name,
Department.Dept_Name
FROM Employee
LEFT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID
UNION
SELECT Employee.Emp_ID,
Employee.Emp_Name,
Department.Dept_Name
FROM Employee
RIGHT JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;
-- NATURAL JOIN
SELECT *
FROM Employee
NATURAL JOIN Department;
Emp_ID	Emp_Name	Dept_Name
101	SHIVA	Computer Science
102	Ananya	Management
103	Karuna	Computer Science
104	Priya	NULL
NULL	NULL	Finance
Dept_ID	Emp_ID	Emp_Name	Dept_Name
1	101	SHIVA	Computer Science
2	102	Ananya	Management
1	103	Karuna	Computer Science
Ready
Light
Wiki

LAB 8:


sql online compiler - Search
MySQL Online Editor
onecompiler.com/mysql
School
queries.sql
MySQL
Run
I/O
AI Agent
103 ms
STDIN
Input for the program (Optional)
Output:
Ready
Light
Wiki
CREATE TABLE Employee (
Emp_ID INT,
Emp_Name VARCHAR(50),
Department VARCHAR(50),
Salary INT
);
INSERT INTO Employee VALUES
(101, 'Shiva', 'IT', 50000),
(102, 'Ananya', 'HR', 40000),
(103, 'Karuna', 'IT', 60000),
(104, 'Priya', 'HR', 45000),
(105, 'Aniva', 'Finance', 55000);
-- GROUP BY
SELECT Department, COUNT(*) AS Number_of_Employees
FROM Employee
GROUP BY Department;
-- GROUP BY with SUM
SELECT Department, SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department;
-- GROUP BY with AVG
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department;
-- HAVING
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 45000;
-- ORDER BY Ascending
SELECT *
FROM Employee
ORDER BY Salary ASC;
-- ORDER BY Descending
SELECT *
FROM Employee
ORDER BY Salary DESC;
-- GROUP BY + HAVING + ORDER BY
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 40000
ORDER BY Average_Salary DESC;
Department	Number_of_Employees
IT	2
HR	2
Finance	1
Department	Total_Salary
IT	110000
HR	85000
Finance	55000
Department	Average_Salary
IT	55000.0000
HR	42500.0000
Finance	55000.0000
Emp_ID	Emp_Name	Department	Salary
102	Ananya	HR	40000
104	Priya	HR	45000
101	Shiva	IT	50000
105	Aniva	Finance	55000
103	Karuna	IT	60000

LAB 9:


sql online compiler - Search
MySQL Online Editor
onecompiler.com/mysql
School
queries.sql
MySQL
Run
I/O
AI Agent
82 ms
STDIN
Input for the program (Optional)
Output:
Ready
Light
Wiki
-- Basic Procedure
CREATE PROCEDURE GetEmployees()
BEGIN
SELECT * FROM Employee;
END //
DELIMITER ;
CALL GetEmployees();
DELIMITER //
CREATE PROCEDURE InsertEmployee(
IN p_id INT,
IN p_name VARCHAR(50),
IN p_dept VARCHAR(50),
IN p_salary INT
)
BEGIN
INSERT INTO Employee
VALUES (p_id, p_name, p_dept, p_salary);
END //
DELIMITER ;
CALL InsertEmployee(103, 'Kiran', 'Finance', 55000);
SELECT * FROM Employee;
DELIMITER //
CREATE PROCEDURE DeleteEmployee(
IN p_id INT
)
BEGIN
DELETE FROM Employee
WHERE Emp_ID = p_id;
END //
DELIMITER ;
CALL DeleteEmployee(103);
SELECT * FROM Employee;

LAB 10:

sql online compiler - Search | MySQL Online Editor | onecompiler.com/mysql

queries.sql | MySQL | Run | I/O | AI Agent | 100 ms

```sql
-- Create Employee table
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(50),
    Salary INT
);

-- Create Log table
CREATE TABLE Employee_Log (
    Log_ID INT AUTO_INCREMENT PRIMARY KEY,
    Action_Type VARCHAR(30),
    Emp_ID INT
);

-- 1. BEFORE INSERT Trigger - Validation
DELIMITER //

CREATE TRIGGER Before_Insert_Employee
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //

DELIMITER ;


-- 2. AFTER INSERT Trigger
DELIMITER //

CREATE TRIGGER After_Insert_Employee
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Log(Action_Type, Emp_ID)
    VALUES ('INSERT', NEW.Emp_ID);
END //

DELIMITER ;
```

STDIN

Input for the program (Optional)

Output:

```
+--------+-------------+--------+
| Log_ID | Action_Type | Emp_ID |
+--------+-------------+--------+
|      1 | INSERT      |    101 |
|      2 | UPDATE      |    101 |
|      3 | DELETE      |    101 |
+--------+-------------+--------+
```

Ready | Light | Wiki

sql online compiler - Search | MySQL Online Editor | onecompiler.com/mysql

queries.sql | MySQL | Run | I/O | AI Agent | 100 ms

```sql
-- 3. BEFORE UPDATE Trigger - Validation
DELIMITER //

CREATE TRIGGER Before_Update_Employee
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //

DELIMITER ;


-- 4. AFTER UPDATE Trigger
DELIMITER //

CREATE TRIGGER After_Update_Employee
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Log(Action_Type, Emp_ID)
    VALUES ('UPDATE', NEW.Emp_ID);
END //

DELIMITER ;


-- 5. BEFORE DELETE Trigger
DELIMITER //

CREATE TRIGGER Before_Delete_Employee
BEFORE DELETE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary > 100000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee with salary above 100000 cannot be deleted';
    END IF;
END //

DELIMITER ;
```

Ready | Light | Wiki

sql online compiler - Search | MySQL Online Editor | onecompiler.com/mysql

queries.sql | MySQL | Run | I/O | AI Agent | 100 ms

```sql
DELIMITER //

CREATE TRIGGER Before_Delete_Employee
BEFORE DELETE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary > 100000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee with salary above 100000 cannot be deleted';
    END IF;
END //

DELIMITER ;


-- 6. AFTER DELETE Trigger
DELIMITER //

CREATE TRIGGER After_Delete_Employee
AFTER DELETE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Log(Action_Type, Emp_ID)
    VALUES ('DELETE', OLD.Emp_ID);
END //

DELIMITER ;


-- Test INSERT
INSERT INTO Employee
VALUES (101, 'Rahul', 50000);

-- Test UPDATE
UPDATE Employee
SET Salary = 60000
WHERE Emp_ID = 101;

-- Test DELETE
DELETE FROM Employee
WHERE Emp_ID = 101;

-- Display log
SELECT * FROM Employee_Log;
```

Ready | Light | Wiki